

Code Explanation

Code Explanation: AWS Glue PySpark ETL Job Test Suite
This is a comprehensive test suite for an AWS Glue PySpark ETL job that processes customer and order data. Here's a clear breakdown:---

****1. Test Setup & Mocking (Lines 1-27)****

****Purpose:**** Prepare the test environment by mocking AWS Glue dependencies.- ****Mocks AWS Glue modules**** (``awsglue``, ``awsglue.context``, ``awsglue.job``, etc.) before importing the actual job code- This allows tests to run locally without AWS Glue runtime dependencies- Imports PySpark, pytest, and the functions being tested from ``main.job``---

****2. Test Fixtures (Lines 30-95)****

****Purpose:**** Create reusable test data for multiple test cases.

`**`spark` fixture:**`

- Creates a local Spark session for testing- Uses ``local[2]`` mode (2 threads) for parallel processing- Session persists across all tests (scope="session")

`**`sample_customer_data` fixture:**`

- Creates 5 sample customer records with schema: custid, name, emailid, region- Clean data with no nulls or duplicates

`**`sample_order_data` fixture:**`

- Creates 5 sample order records with schema: orderid, itemname, priceperunit, qty, date, custid- Uses Decimal type for prices (financial precision)- Orders link to customers via custid

`**`sample_customer_with_nulls` fixture:**`

- Creates customer data with data quality issues: - NULL values in various columns - String "Null" values - Duplicate records- Used to test data cleaning functions---

****3. Configuration Tests (Lines 98-117)****

`**`test_get_job_parameters`:**`

- Verifies job parameter loading works correctly- Checks that required parameters exist (customer path, order path, database name)- Validates default database name is "gen_ai_poc_databrickscoe"

`**`test_validate_s3_path`:**`

- Tests S3 path validation logic- ****Valid paths:**** `s3://`, `s3a://` protocols- ****Invalid paths:**** empty strings, None, local paths, `file://` URLs---

****4. Data Cleaning Tests (Lines 120-169)****

****`test\_clean\_dataframe\_removes\_nulls`:****

- Verifies NULL values are removed from dataset- Expects only 2 complete records from the 6-record input- Checks every column for remaining nulls

****`test\_clean\_dataframe\_removes\_string\_null`:****

- Ensures string literal "Null" values are removed- Validates no "Null" strings remain in any column

****`test\_clean\_dataframe\_removes\_duplicates`:****

- Confirms duplicate records are eliminated- Verifies distinct count equals total count after cleaning

****`test\_clean\_dataframe\_with\_none\_input`:****

- Tests graceful handling of None input- Should return None without crashing---

****5. SCD Type 2 Tests (Lines 172-194)****

****Purpose:**** Test Slowly Changing Dimension Type 2 functionality.

****`test\_add\_scd2\_columns`:****

- Verifies 4 SCD2 columns are added: isactive, startdate, enddate, opts- Checks all records are marked as active (isactive=True)- Validates enddate is NULL for active records (standard SCD2 pattern)

****`test\_add\_scd2\_columns\_with\_none\_input`:****

- Tests error handling for None input---

****6. Change Detection Tests (Lines 197-220)****

****Purpose:**** Test incremental data processing logic.

****`test\_detect\_customer\_changes\_no\_baseline`:****

- Tests scenario with no existing baseline data- All 5 records should be detected as new

****`test\_detect\_customer\_changes\_with\_baseline`:****

- Tests change detection with existing baseline (C001, C002)- Should detect 3 new customers (C003, C004, C005)- Simulates incremental load scenario

****`test\_detect\_customer\_changes\_with\_none\_input`:****

- Tests error handling---

****7. Order Summary Tests (Lines 223-245)****

****Purpose:**** Test customer-order join logic.

****`test\_create\_order\_summary`:****

- Verifies customer and order data are joined correctly- All 5 orders should match with customers- Validates all expected columns exist after join

****`test\_create\_order\_summary\_with\_missing\_input`:****

- Tests both None scenarios (missing customers or orders)---

****8. Aggregate Spend Tests (Lines 248-289)****

****Purpose:**** Test business logic for calculating customer spending.

****`test\_calculate\_customer\_aggregate\_spend`:****

- Verifies aggregation by customer name and date- John Doe should have 2 records (orders on different dates)- Checks required output columns exist

****`test\_calculate\_customer\_aggregate\_spend\_amounts`:****

- Validates actual calculation accuracy- John Doe's Jan 15 spending should equal \$1200.00 (1 laptop)- Tests financial calculation precision

****`test\_calculate\_customer\_aggregate\_spend\_with\_none\_input`:****

- Tests error handling---

****9. Data Reader Tests (Lines 292-316)****

****Purpose:**** Test CSV reading functionality.

****`test\_data\_reader\_read\_csv`:****

- Mocks Spark's CSV reader- Verifies DataReader can read CSV files from S3- Tests method chaining (format → option → load)

****`test\_data\_reader\_read\_csv\_invalid\_path`:****

- Ensures invalid paths return None instead of crashing---

****10. Data Writer Tests (Lines 319-380)****

****Purpose:**** Test data writing to various formats.

****`test\_data\_writer\_write\_dataframe\_parquet`:****

- Tests Parquet format writing- Mocks internal `_write_parquet` method- Verifies method is called correctly

****`test\_data\_writer\_write\_dataframe\_hudi`:****

- Tests Apache Hudi format writing (for ACID transactions)- Validates Hudi configuration (table name, record key, precombine field)

****`test\_data\_writer\_write\_dataframe\_invalid\_path`:****

- Tests invalid path handling (should return False)

****`test\_data\_writer\_write\_dataframe\_empty\_df`:****

- Tests empty DataFrame handling (should return False)

****`test\_data\_writer\_write\_to\_catalog`:****

- Tests writing to AWS Glue Data Catalog- Mocks DynamicFrame conversion- Validates catalog registration attempt---

****11. Column Normalization Test (Lines 383-401)****

****Purpose:**** Ensure consistent column naming.- Tests that mixed-case column names (CustId, Name) are normalized to lowercase (custid, name)- Important for consistent data processing across systems---

****12. End-to-End Integration Test (Lines 404-434)****

****Purpose:**** Validate complete pipeline flow.****Pipeline Steps:****1. ****Clean data**** - Remove nulls, duplicates2. ****Add SCD2 columns**** - Add tracking columns3. ****Create order summary**** - Join customers and orders4. ****Calculate aggregate spend**** - Group by customer and date****Validations:****- Each step produces non-null output- Final output has 5 joined records- Required columns exist at each stage- Data flows correctly through entire pipeline---

****Key Testing Patterns Used:****

1. ****Fixtures**** - Reusable test data2. ****Mocking**** - Isolate code from AWS dependencies3. ****Assertions**** - Verify expected behavior4. ****Edge cases**** - Test None inputs, empty data, invalid paths5. ****Integration testing**** - Test complete workflow6. ****Data quality validation**** - Verify cleaning, deduplication, null handlingThis test suite ensures the ETL job handles normal operations, edge cases, and error conditions correctly before deployment to AWS Glue.